



3D MODELING OF DETACHED TIN WHISKERS

OPERATIONAL READINESS REPORT

MECH 4240 Summer 2025

Manager: Cameron Brock

Scribe: Austin Hill

Daniel Wheeler, Roy Chen

Sponsor: Mr. Stephen Wells

Technical Advisor: Dr. George Flowers

TABLE OF CONTENTS

<u>Chapter</u>	<u>page</u>
TABLE OF CONTENTS.....	2
LIST OF TABLES	4
LIST OF FIGURES	5
LIST OF ABBREVIATIONS.....	7
INTRODUCTION	8
DESIGN SPECIFICATIONS	10
BACKGROUND INFORMATION	11
Tin Whisker Information	11
Simulation Information: Monte Carlo & Unity	11
Previous Groups' Work: The Unity Simulator.....	16
DESIGN ELEMENTS AND SELECTION.....	23
Electrostatic Attraction Forces	23
Simulation Profile Creation	23
Falcon 9 Launch Demonstration Profile.....	25
PCB Model Importing	27
Subfunction Decision Making	35
DEVELOPMENT, TESTING, AND RESULTS	37
Simulation Profiles Addition	37
Vibration and Shock Time Activation.....	37
Time-Dependent Acceleration.....	38
Acceleration Model Development.....	38
Simulation Acceleration Implementation.....	40
Profile Saving	41
Adding Electrostatic Attraction	41
Equation/Setup	41
Constant Position/Magnitude	41
Changing Position/Magnitude	42
Heatmap Improvement	44
Testing	44
Discussion.....	46
Conclusion.....	47
PCB Import.....	48

Testing Phase for Importing PCB.....	48
APPENDIX A.....	53
APPENDIX B.....	55

LIST OF TABLES

<u>Table</u>	<u>page</u>
Table 1: Example Excel Data of All Whiskers from a Simulation.....	13
Table 2: Example Excel Data of Bridged Whiskers from a Simulation	14
Table 3: Example Data of Bridged Pair Data from Simulation	15
Table 4: Subfunction Decision Matrix.....	35

LIST OF FIGURES

<u>Figure</u>	<u>page</u>
Figure 1: Example Chart of the Bridged Conductor Pairs from a Simulation.....	16
Figure 2: Example Circuit Board in Unity.....	17
Figure 3: Whisker Properties UI (Dimensions, Distribution, Whisker Number, and Material Selection Interface)	18
Figure 4: Whisker Properties UI (Spawn Point and Spawn Area Interface)	19
Figure 5: Board Control and MCS UI (Gravity, Iterations, Save/Load, and Barriers).....	20
Figure 6: Board Control and MCS UI (Board Rotation/Spin, Shock, and Vibration Interface)....	21
Figure 7: Critical Pairs UI.....	21
Figure 8: Simulation Confirmation UI.....	22
Figure 9: Output Tab.....	22
Figure 10: Falcon 9 Acceleration Profile.....	25
Figure 11: Sinusoidal Vibration of Falcon 9 Launch.....	26
Figure 12: Shock Events of Falcon 9 Launch.....	26
Figure 13: PB CAD File in KiCAD.....	27
Figure 14: PB Export Interface	28
Figure 15: Disabled Fuse Shapes Configuration	29
Figure 16: Fusion360 PCB Model	29
Figure 17: Exporting as .obj File	30
Figure 18: PCB .obj File in Unity.....	31
Figure 19: Unity Inspector Tab.....	32
Figure 20: Unity Model Panel.....	32
Figure 21: Unity In-Simulation Model	33
Figure 22: Whiskers on Imported PCB.....	34

Figure 23: Vibration and Shock Time based Inputs	38
Figure 24: Time-dependent Acceleration Model.....	40
Figure 25: Whisker Properties Tab with Electrostatic Attraction.....	43
Figure 26: Electrostatic Attraction UI.....	43
Figure 27: Whisker Model Team Gantt Chart	55

LIST OF ABBREVIATIONS

PCB	Printed Circuit Board is the compiled structure of conductive and insulating layers with electrical components attached to the surface through soldering forming an electrical circuit.
CCA	Circuit Card Assembly is another term for a Printed Circuit Board. An electrical circuit of conductive and insulating material with electrical components.

CHAPTER 1

INTRODUCTION

If a device has an electronic function, it almost certainly has a printed circuit board (PCB) inside—powering everything from smartphones and medical devices to fighter jets and spacecraft. However, tin whiskers—microscopic metallic filaments that grow spontaneously on tin-coated surfaces—pose a serious reliability risk to these systems. Whiskers can physically span the tiny gaps between conductive elements on a PCB, causing unintended electrical connections that lead to short circuits. These shorts can disrupt circuit operation, trigger intermittent failures, or even cause catastrophic system malfunctions.

Tin whiskers have been implicated in major electronic failures over the decades, including disruptions in satellite systems, missile defense technology, and NASA’s space programs. Historically, the addition of lead (Pb) to tin coatings suppressed whisker growth, but modern environmental regulations have drastically reduced lead usage, increasing the susceptibility of electronic components to whisker-induced shorts. In addition to environmental and chemical challenges, the miniaturization of electronic circuits has exacerbated the problem by reducing the spacing between conductive traces, making inadvertent bridging—and thus electrical shorts—even more likely. The effectiveness of current mitigation strategies, such as conformal coatings and material substitutions, have had limited success but may not always be a feasible solution.

To address these challenges, this project’s aim was to develop a predictive simulation tool that evaluates conductive bridging risks by analyzing tin whisker formation, growth characteristics, and movements under various environmental conditions. The tool, built on the

Unity Game Engine, not only models whisker behavior on PCBs but also incorporates multiple challenges faced by modern electronics:

- **Electrostatic Attraction:** Simulating how whiskers are drawn to conductors by electrostatic charges causing shorts by unintentional electrical bridges.
- **Dynamic Environmental Factors:** Integrating real-life conditions such as acceleration, vibration, and shock scenarios such as PCBs carried on a rocket during launch based on data from the SpaceX Falcon User's Guide (2021) (specifically Section 4.3, which provides critical data for Falcon 9 and Falcon Heavy rockets).
- **PCB Model Import:** Facilitating the import of PCB designs from Fusion 360, Altium, and Blender to support practical and adaptable testing across a diverse range of board designs.

By allowing users to input environmental parameters that mirror those experienced during real operations and by accurately simulating the electromechanical behavior of tin whiskers, our tool works to deliver clear, actionable risk assessments. This powerful simulation aims to help PCB designers mitigate whisker-induced short circuits, thereby improving the reliability of electronic components across mission-critical industries.

CHAPTER 2

DESIGN SPECIFICATIONS

This project must operate within a set of constraints defined by the limitations of the previous Unity simulation models created by the Auburn groups (Team 1 & 2). Among these constraints is the requirement that the simulation output be clear, well-organized, and self-explanatory to facilitate practical data analysis.

- **Software Development Constraints**

- The software tool must be developed using a single coding and scripting language to maintain consistency and ease of development.
- The Unity game engine must be used for advanced 3D modeling and visualization.

- **System Compatibility and Modularity**

- The software must be compatible with existing system models and data formats used in previous development efforts.

- **Accuracy and Performance**

- The refined model must accurately compute the probability of mechanical bridging of detached tin whiskers.
- The software must evaluate the impact of bridging on the electronic behavior of the CCA as per the reference document specifications.

- **Environmental and Physical Constraints**

- The simulation must account for external dynamic loads (e.g., handling, transportation, operational service environments) that may cause tin whiskers to detach.
- The modeling must realistically represent tin whisker behavior within electronic enclosures under relevant environmental conditions.

CHAPTER 3

BACKGROUND INFORMATION

Tin Whisker Information

Tin whiskering causes large problems with circuits that have tin plating due to the whiskers potentially bridging the gap in between two circuits, causing it to short or otherwise fail. Whiskering is noted as being even more dangerous not just because of the common use of tin- the metal most prone to whiskering- but also because it is not fully understood what causes whiskers to grow. Common theories are residual stresses within the plating, intermetallic information, externally applied compressive stresses, bending or stretching of the plating, scratches or other damages, or potentially coefficient of thermal expansion mismatches in the plating material and its substrate, though there has been conflicting evidence for these factors. This unpredictability adds to the danger of tin whiskers because it is more difficult to prepare for them if it is not certain how or when they will form.

Simulation Information: Monte Carlo & Unity

The nature of tin whisker growth makes a Monte Carlo simulation approach ideal for predicting bridging events. In a Monte Carlo simulation, thousands of iterations are run with random input variables (such as whisker length, diameter, and orientation), and the outcomes are statistically analyzed. This randomness captures the variability observed in real-life whisker formation processes. Key aspects include:

- **Monte Carlo Equations:**

The simulation relies on probabilistic models to determine whisker properties. For example, whisker length and diameter might be generated using normal or lognormal

distributions defined by user-specified mean (μ) and standard deviation (σ) values. An equation from probability theory, such as:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

is used to generate each whisker's dimensions. These equations are coded in C# within the Unity environment to ensure that each iteration reflects realistic, random whisker behavior.

- **Unity & C# Implementation:**

Unity, powered by C#, provides the ideal platform for 3D simulations. The C# scripts control the generation and movement of whiskers based on user input and Monte Carlo-generated parameters. For instance, the scripts calculate the probability of whisker bridging events by updating the whiskers' positions and checking collisions with conductor regions on the virtual PCB model in real time.

- **Excel Output:**

After each simulation run, key parameters such as whisker length, width, resistivity, and the iteration number when bridging occurs are exported to an Excel/CSV file, examples of which are shown below in Tables 1-3 and Figure 1. Table 1 shows the example information for data gathered from all whiskers that are found in the simulation, whereas Table 2 shows the same information, but only including the whiskers that have bridged and adding information about which conductors were bridged. Table 3 shows example information for circuit board data in the simulation, being which pairs were bridged, how many times that specific pair of conductors were bridged, and what percentage of bridging that pair accounted for, with Figure 1 being the bar chart (with a line graph) of

that bridged pair data. These outputs allow engineers to perform statistical analyses (e.g., averaging values, calculating standard deviations, and determining bridging probability metrics) and compare the simulation data with experimental results.

Table 1: Example Excel Data of All Whiskers from a Simulation

All Whiskers					
Whisker #	Length (um)	Width (um)	Resistance (ohm)	Iteration	
1	3999.173	99.80888	0.05571452	1	
2	4001.038	99.66998	0.05589598	1	
3	3999.219	100.0819	0.05541163	1	
4	3998.486	99.58105	0.05596013	1	
5	4000.987	99.94893	0.05558369	1	
6	4001.222	99.6096	0.05596633	1	
7	4001.122	100.397	0.05509054	1	
8	3997.827	99.39907	0.05615598	1	
9	4000.042	98.88059	0.05677787	1	
10	4000.345	100.0352	0.05547893	1	
11	3999.434	99.8668	0.05565355	1	
12	3999.535	99.61581	0.05593577	1	
13	4001.247	100.4386	0.05504657	1	
14	3998.718	101.8553	0.05349212	1	
15	3999.51	99.97604	0.05553305	1	
16	3999.263	98.92239	0.05671884	1	
17	3998.384	100.3519	0.05510231	1	
18	4001.204	99.99858	0.05553153	1	
19	3997.663	99.51509	0.0560228	1	
20	4000.899	101.7102	0.05367418	1	
21	3999.334	99.06388	0.05655794	1	
22	3998.69	100.4335	0.05501707	1	
23	4000.97	101.0273	0.05440324	1	
24	3999.092	99.87579	0.05563877	1	
25	4000.326	99.90173	0.05562704	1	
26	4000.667	99.12151	0.05651102	1	
27	3998.76	99.49951	0.05605573	1	
28	4000.901	99.77075	0.0557812	1	
29	3999.882	99.64478	0.05590808	1	
30	4000.656	99.45628	0.05613108	1	
31	4000.083	99.90025	0.05562531	1	
32	4000.989	99.59743	0.05597675	1	
33	3999.779	100.4938	0.05496603	1	
34	4000.224	99.42568	0.05615957	1	
35	4000.032	99.77779	0.05576123	1	
36	3998.6	99.86333	0.05564581	1	
37	3999.417	99.0532	0.05657131	1	
38	3997.596	98.82497	0.05680704	1	
39	4001.593	100.0834	0.05544285	1	
40	3998.891	100.3848	0.05507315	1	
41	3999.557	100.6498	0.05479272	1	
42	3999.256	99.68345	0.05585598	1	
43	3999.522	99.14796	0.05646471	1	
44	3999.368	99.39328	0.05618417	1	
45	4001.699	99.41785	0.05618913	1	
46	3998.299	100.9744	0.05442385	1	
47	3999.789	100.4135	0.05505411	1	
48	3999.772	100.2194	0.05526733	1	
49	4000.189	100.1858	0.05531015	1	
50	4000.815	99.78518	0.05576389	1	
51	4000.195	99.92706	0.05559701	1	

(Note: Tables 1-3 are shortened for brevity, only containing roughly a quarter of what would actually be shown in an Excel file.)

Table 2: Example Excel Data of Bridged Whiskers from a Simulation

Bridged Whiskers									
Whisker #	Length (um)	Width (um)	Resistance (ohm)	Iteration	Conductor 1	Conductor 2	Bridged Pair		
318	3272.759	51.13706	0.1736917	1	GND	Q1	GND + Q1		
352	4102.102	55.02091	0.1880561	1	GND	Q1	GND + Q1		
26	4421.396	53.90409	0.2111799	1	Q1	GND	Q1 + GND		
43	3747.569	56.62005	0.1622355	1	R28	NetD3_1	R28 + NetD3_1		
56	2372.752	45.65596	0.1579791	1	C4	NetC4_1	C4 + NetC4_1		
198	6977.418	59.15441	0.2767306	1	Q2	NetQ2_2	Q2 + NetQ2_2		
117	6817.568	51.01569	0.3635453	1	GND	NetQ1_1	GND + NetQ1_1		
9	6449.581	33.88324	0.7796476	1	NetR11_2	C2	NetR11_2 + C2		
381	2518.593	63.03785	0.08796133	1	R30	NetR29_2	R30 + NetR29_2		
368	1159.62	62.50555	0.04119219	1	R11	NetR11_2	R11 + NetR11_2		
66	2761.829	66.48966	0.08670381	1	Q1	NetQ1_2	Q1 + NetQ1_2		
274	3592.661	43.09268	0.268501	1	NetR11_2	R15	NetR11_2 + R15		
260	3992.121	59.58002	0.156077	1	NetR7_2	NetC1_1	NetR7_2 + NetC1_1		
240	2707.194	62.07954	0.09748974	1	R9	NetR8_2	R9 + NetR8_2		
387	7638.089	50.51381	0.4154331	1	GND	C6	GND + C6		
30	6010.599	65.75401	0.1929345	1	C1	R13	C1 + R13		
1	2921.423	57.96461	0.1206716	1	R14	NetR13_2	R14 + NetR13_2		
2	5753.229	72.59191	0.1515206	1	NetC5_1	C5	NetC5_1 + C5		
178	5572.637	46.1104	0.3637472	1	GND	NetQ1_1	GND + NetQ1_1		
49	1550.037	53.73452	0.07450269	1	D4	NetD4_1	D4 + NetD4_1		
341	2707.687	50.81097	0.1455526	1	NetD4_1	NetR35_2	NetD4_1 + NetR35_2		
351	1349.875	62.37588	0.04815006	1	R17	NetC4_2	R17 + NetC4_2		
226	3739.276	46.04363	0.2447852	1	Q1	NetQ1_1	Q1 + NetQ1_1		
294	6577.195	72.02562	0.1759557	1	NetC5_1	R33	NetC5_1 + R33		
235	3636.99	53.09809	0.1790279	1	R35	NetR35_2	R35 + NetR35_2		
184	4313.99	59.50294	0.1690981	1	R19	NetD2_1	R19 + NetD2_1		
377	3206.46	84.73109	0.06198359	1	R10	NetC2_1	R10 + NetC2_1		
345	4330.444	58.78375	0.173922	1	R17	NetR16_2	R17 + NetR16_2		
300	4861.899	66.54636	0.152368	1	R12	NetR11_2	R12 + NetR11_2		
211	2487.99	53.98627	0.1184728	1	R21	NetR20_2	R21 + NetR20_2		
309	6000.833	70.68879	0.166666	1	R29	NetL2_2	R29 + NetL2_2		
227	5880.828	59.88322	0.2275961	1	R7	NetR7_2	R7 + NetR7_2		
313	10783.84	71.07935	0.296226	2	Q2	Q1	Q2 + Q1		
133	12380.76	54.64743	0.5753661	2	C6	NetC6_2	C6 + NetC6_2		
11	2026.925	50.15923	0.1118079	2	R2	NetQ1_2	R2 + NetQ1_2		
47	3582.021	38.31915	0.338558	2	R3	NetQ2_2	R3 + NetQ2_2		
152	3644.483	51.95587	0.1873714	2	GND	NetQ1_1	GND + NetQ1_1		
365	4857.57	42.19027	0.3787312	2	R22	NetR22_2	R22 + NetR22_2		
34	3605.776	59.40986	0.1417811	2	C2	NetC2_2	C2 + NetC2_2		
39	4817.269	55.73123	0.2152485	2	R4	R3	R4 + R3		
269	2054.268	35.93402	0.2207914	2	NetD4_1	NetR35_2	NetD4_1 + NetR35_2		
69	5080.737	56.53075	0.2206448	2	GND	NetQ1_1	GND + NetQ1_1		
114	4482.145	56.50871	0.1948012	2	R32	NetR31_2	R32 + NetR31_2		
228	1390.464	57.37443	0.05862186	2	R33	NetC5_1	R33 + NetC5_1		
314	3803.127	35.03756	0.4299419	2	NetD4_2	D4	NetD4_2 + D4		
381	3576.365	53.6446	0.1724751	2	D2	NetD2_2	D2 + NetD2_2		
309	1670.751	46.67436	0.1064368	2	GND	NetQ1_1	GND + NetQ1_1		
390	2365.701	73.41203	0.06092031	2	NetR20_2	R21	NetR20_2 + R21		
144	3395.771	55.70953	0.1518505	2	R12	NetR11_2	R12 + NetR11_2		
373	2875.217	54.53157	0.1341872	2	R33	NetC5_1	R33 + NetC5_1		
155	15254.52	51.42295	0.8006102	2	C5	NetD3_1	C5 + NetD3_1		

Table 3: Example Data of Bridged Pair Data from Simulation

Bridged Pair Analysis			
Bridged Pairs		Count of Bridged Pair	Distinct Count of Iteration
GND + NetQ1_1		53	83.33%
R28 + NetD3_1		15	43.33%
R6 + NetC1_2		14	33.33%
GND + Q1		14	36.67%
Q1 + GND		14	33.33%
R16 + NetR16_2		11	30.00%
D2 + NetD2_1		9	30.00%
R2 + NetQ2_1		9	30.00%
R30 + NetR29_2		8	26.67%
R11 + NetC2_1		7	23.33%
D2 + NetD2_2		7	23.33%
R29 + NetL2_2		6	16.67%
C1 + NetC1_2		6	16.67%
R38 + NetR37_2		6	16.67%
C4 + NetC4_1		6	20.00%
C2 + NetC2_2		6	20.00%
R19 + NetD2_1		6	20.00%
NetR29_2 + R29		6	16.67%
L2 + NetL2_1		5	16.67%
C5 + NetC5_1		5	16.67%
R35 + NetD4_1		5	13.33%
NetR11_2 + R11		5	16.67%
R5 + NetD1_1		5	16.67%
R3 + NetR3_2		4	13.33%
Q2 + NetQ2_2		4	13.33%
NetR19_2 + R19		4	13.33%
GND + NetR38_2		4	13.33%
R39 + GND		4	13.33%
NetR11_2 + R15		4	10.00%
NetL2_1 + NetD3_1		4	13.33%
R16 + NetR8_1		4	13.33%
Q2 + NetQ2_1		3	10.00%
C6 + NetC6_2		3	10.00%
C4 + NetC4_2		2	6.67%
NetQ2_1 + R2		2	6.67%
R8 + NetR8_1		2	6.67%
Q1 + NetQ1_1		2	6.67%
NetR9_2 + R10		2	6.67%
NetR16_2 + R17		2	6.67%
C5 + NetC5_2		2	6.67%
NetR29_2 + R30		2	6.67%
R12 + NetR12_1		2	6.67%

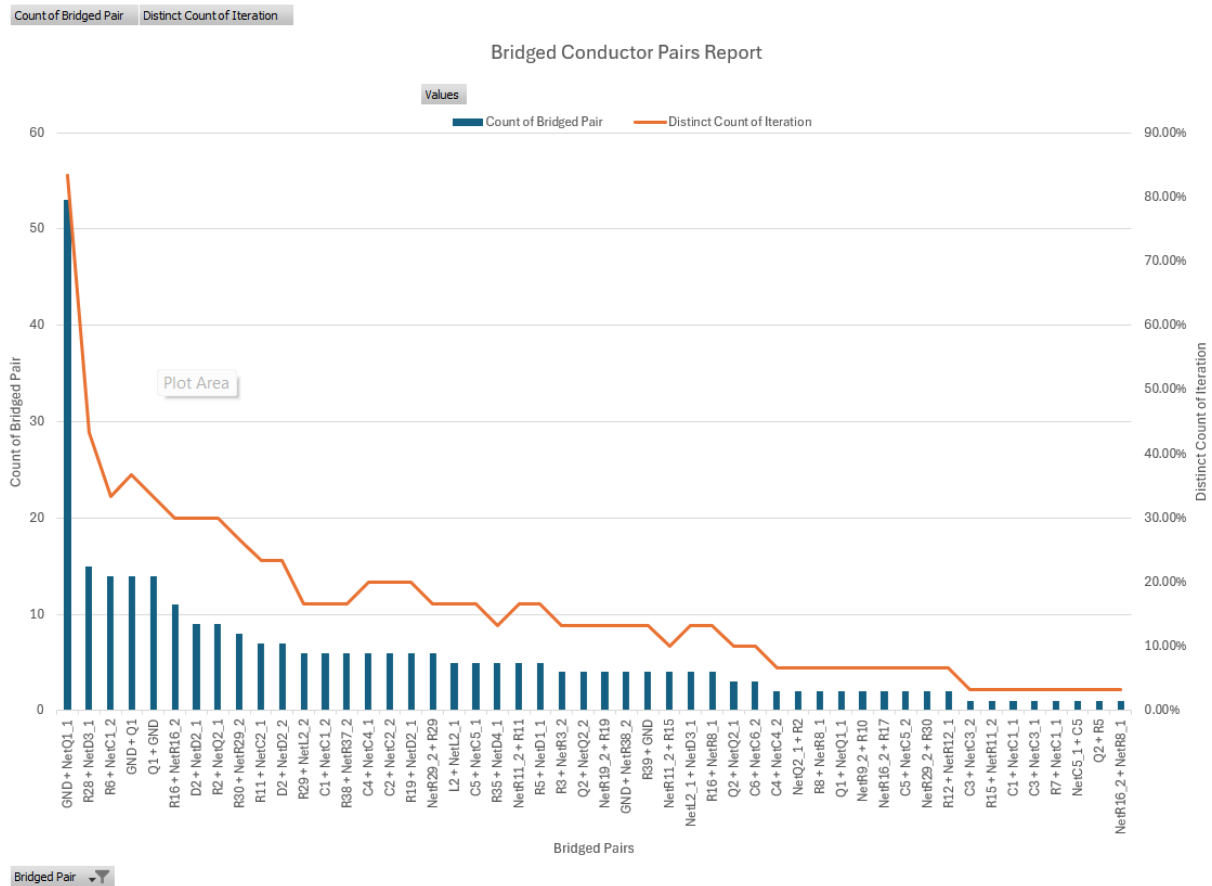


Figure 1: Example Chart of the Bridged Conductor Pairs from a Simulation

Previous Groups' Work: The Unity Simulator

The previous group had designed a Monte Carlo type of simulation in Unity, where a 3D model of a circuit board can be imported. An example shown below in Figure 2.

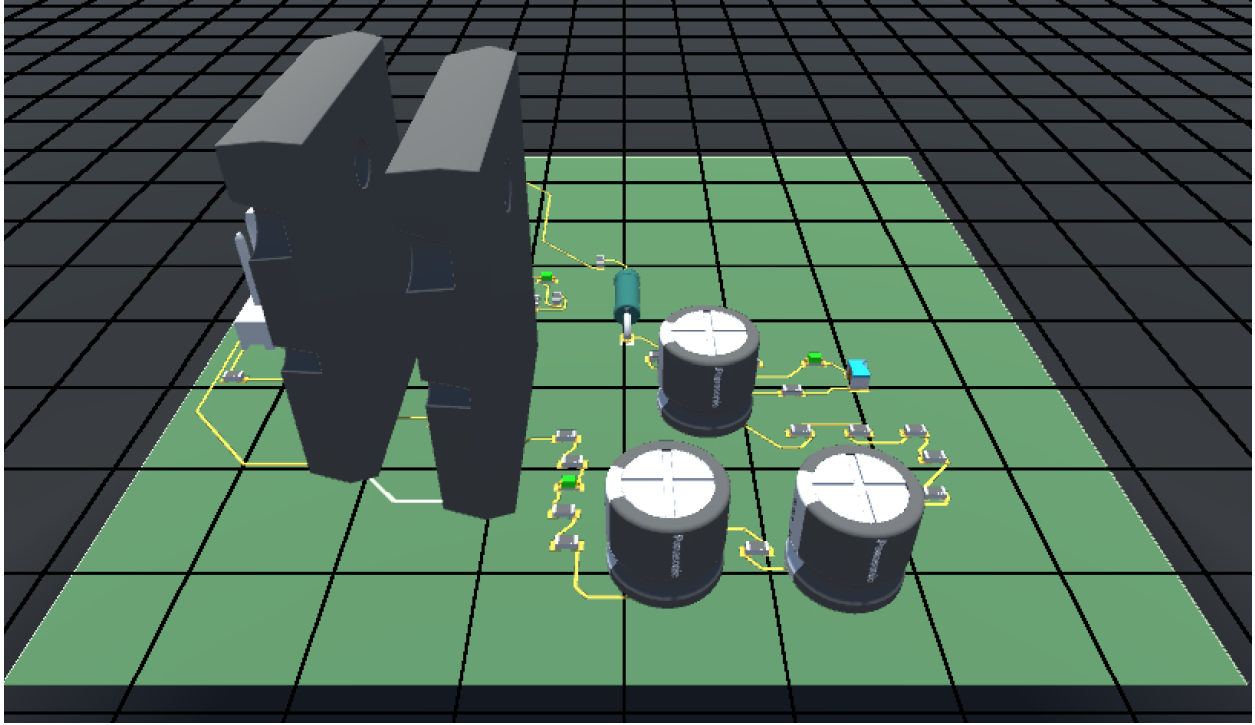


Figure 2: Example Circuit Board in Unity

In this simulator, information can be entered via user inputs. Therefore, the whisker properties, the board's movement, and critical pairs (places on the circuit board where the whiskers can cause bridging) can all be changed by the user. Figure 3 (shown below) shows the user interface (UI) for changing the whisker properties. This includes the length and diameter of the whiskers (Length Mu and Width Mu respectively) as well as their corresponding standard deviation (Length Sigma and Width Sigma), all in micrometers. The UI also includes the number of whiskers that will be produced in a simulation, the material of the whisker (Whisker Material, choosing between Tin, Zinc, or Cadmium), and the distribution type of the generation of whiskers (Distribution Select, choosing between Lognormal and Normal). The Conductive Material Selection allows the user to input a name of a material, and the program will search for components of the board that match the material to label as Conductive.

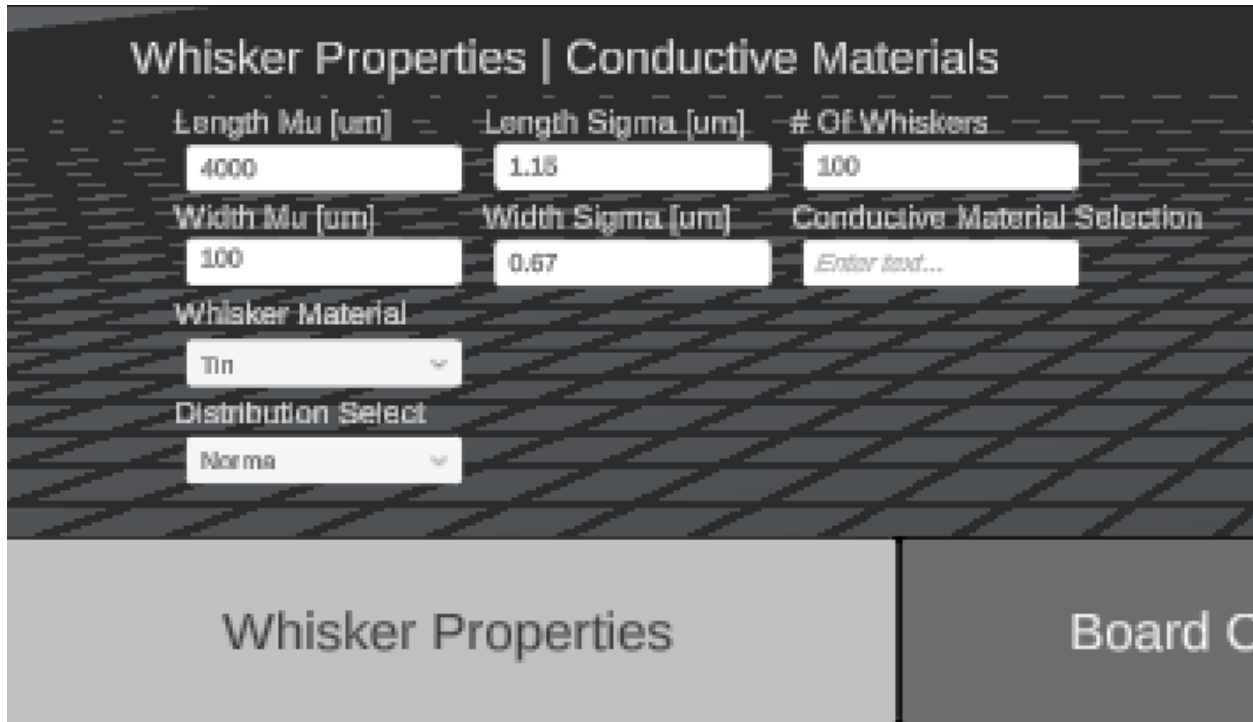


Figure 3: Whisker Properties UI (Dimensions, Distribution, Whisker Number, and Material Selection Interface)

(Note: The pictures/figures given have text that is slightly fuzzy due to it being zoomed in on through Unity.)

Figure 4 (shown below) shows the next part of the Whisker Properties user interface. Whiskers, normally, will spawn randomly in a designated invisible “box” above the circuit board. This UI allows the user to input the edges of the box (Whisker Spawn Point X, Y, Z), as well as the location of the center of the box (X, Y, Z Coordinate) (units are in centimeters). There is a toggleable option to show the invisible “box” where the whiskers will spawn (Whisker Spawn Area).

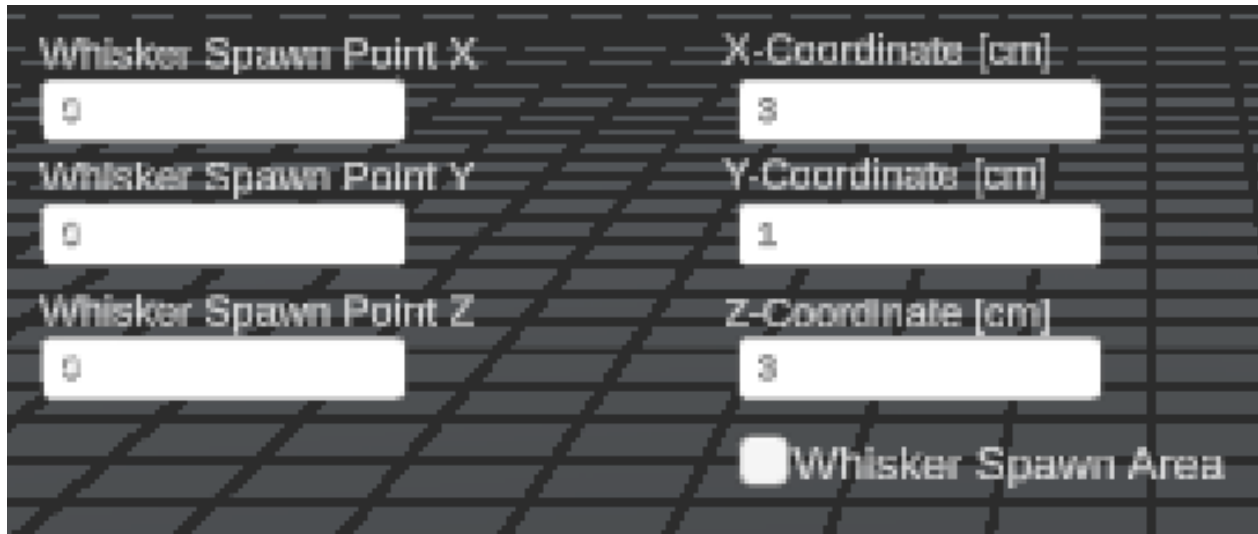


Figure 4: Whisker Properties UI (Spawn Point and Spawn Area Interface)

Figure 5 (shown below) shows the board's properties and the Monte Carlo Simulation (MCS) Properties user interface. The gravity can be changed (Select Gravity, options being Earth, Mars, Moon, and Custom [so the user can input their own gravity constant]). The number of iterations can be changed (# of Iterations), and the data export files can have their name and location changed here (Enter Directory and File Name), with Save and Load Inputs allowing the user to save their inputs and load them at a later date. The Heatmap- a way to show where the whiskers are most likely to congregate- can be toggled here, as well as the option to add walls and ceiling to the board (Add Walls, Wall Height (cm), and toggleable Ceiling).

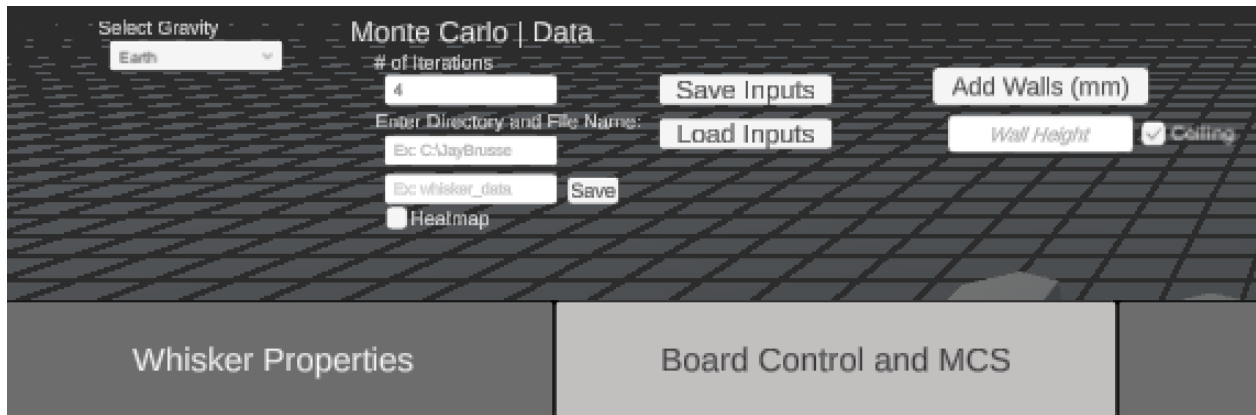


Figure 5: Board Control and MCS UI (Gravity, Iterations, Save/Load, and Barriers)

Figure 6 (shown below) shows the user interface for the ability to rotate, spin, shock, or vibrate the board. Inputs for rotating the board (X, Y, Z, in degrees), spinning the board (X, Y, Z, in degrees per second) is shown, as well as the option give the board a sudden physical jolt with an inputted magnitude (Amplitude, in Newtons) and direction (dropdown box of X, Y, Z), which can be set to happen automatically at the start of each iteration of the simulation (toggleable Use In box). For a constant series of shocks for a specified time, Vibration can also be set, changing its magnitude (Amplitude, in Newtons), speed of the vibration (Frequency, in Hz), the duration (in sec), and direction (dropdown box of X, Y, Z), which can be set to happen automatically at the start of each iteration of the simulation (toggleable Use In Simulation box).



Figure 6: Board Control and MCS UI (Board Rotation/Spin, Shock, and Vibration Interface)

Figure 7 (shown below) shows the user interface for setting up critical pairs. The user can choose two conductors from their Printed Circuit Board (PCB) to act as “critical pairs”, which is a user defined term for particularly important conductors. Although this does not change anything directly in the simulation, the output report and the Excel analysis file that accompanies this will have specific output statistics regarding the pairs.

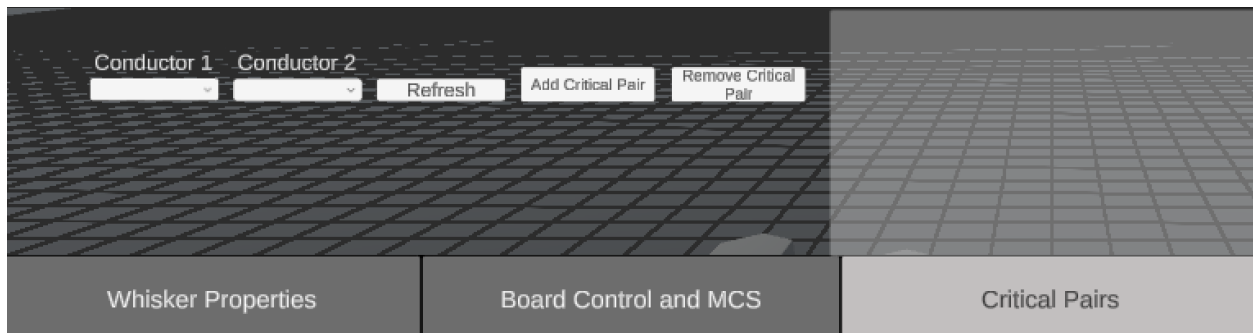


Figure 7: Critical Pairs UI

Figure 8 (shown below) shows the user interface for running the simulation. They can Spawn Whiskers (effectively spawning an iteration of the simulation as a test), Reload (clears out any whiskers and resets the board to match the current UI), Reset Gravity (also allows the whiskers/data storage to be changed to different parameters), Start or Reset the Simulation based on the parameters given, Hide UI/Grid, Raise and Lower Grid (the grid is there to help the user

visualize dimensions, as the grids are 1x1 cm), and Screenshot the simulation with no UI elements (and be able to screenshot automatically after each iteration).

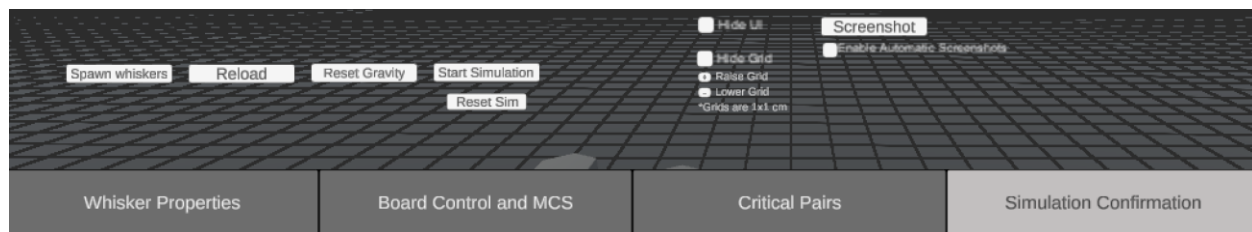


Figure 8: Simulation Confirmation UI

Finally, Figure 9 (shown below) shows the user interface for loading their CSV file, giving information about the bridged whiskers. This information includes Whisker Number in the iteration, their length and diameter (micrometers), resistance (ohms), which iteration they bridged in, and which two conductors bridged the whisker.

Whisker #	Length (um)	Diameter (um)	Resistance (ohm)	Iteration	Conductor 1	Conductor 2
378	148.9126	4.304008	1.115635	1	C4	NetC4_1
31	41.50618	11.91091	0.04060309	1	R18	NetI1_1
103	278.3868	3.647605	2.90382	1	Q1	GND
212	104.031	14.90167	0.06501736	1	Q1	GND
169	1813.943	2.996243	28.04181	1	Q1	GND
270	55.00698	5.558792	0.247055	1	C4	NetC4_1
258	905.7411	5.917198	3.590116	1	O6	NetO6_1
135	637.425	6.627314	2.014144	1	R22	NetR22_2
331	64.33311	3.990573	0.5606614	1	R29	NetI2_2
82	42.49613	5.980281	0.1649084	1	R38	NetR37_2
342	1132.683	6.504404	3.715611	1	R16	NetR16_2
29	182.2725	1.421211	12.52393	2	C5	NetC5_1
318	676.824	1.853635	27.33783	2	R16	NetR8_1
122	591.7322	5.691544	2.535138	2	R35	NetD4_1
190	1756.99	2.292859	46.38216	2	R3	NetR3_2
113	140.4265	4.484106	0.9632461	2	R35	NetD4_1
238	1827.578	2.027065	61.72727	2	D4	NetD4_2
239	1329.099	2.105669	41.60197	3	NetD4_1	D4
284	57.70119	1.937914	2.132319	3	R29	NetI2_2
100	1075.808	2.414742	25.60531	3	GND	NetQ1_1
211	13.09958	8.925994	0.02281817	3	R29	NetR29_2
86	703.0936	5.86483	2.836866	3	R2	NetQ2_1

Figure 9: Output Tab

CHAPTER 4

DESIGN ELEMENTS AND SELECTION

To enhance the predictive simulation tool for tin whisker–induced electrical failures, we have refined our design elements into three primary areas that align with our project’s requirements. The overall simulation architecture, illustrated in Figure 1, highlights the new modules we have introduced.

Electrostatic Attraction Forces

The first element, electrostatic attraction between whiskers, quantifies how electrical forces influence whisker deflection and increase the chance of bridging. Research, such as that by Hilty et al., has shown that even small deflections from applied voltages may cause whiskers to contact adjacent conductors, leading to intermittent shorts. In our implementation, the software computes these forces using simplified Coulomb-based equations specifically adapted for slender filaments, all within Unity’s C# environment. This allows for adjustments in whisker geometry, voltage levels, and material properties.

A dedicated user panel enables the specification of voltage or electrostatic field parameters, and the real-time 3D view (see Figure 2) displays whiskers deflecting under these forces, which helps to identify potential bridging points. This module is integrated in C#, ensuring it smoothly merges with the existing whisker-generation and collision-detection logic. Its use of validated electrostatic equations adds realism to our predictions and models a key real-world phenomenon (electrical bias) that can accelerate whisker bridging in sensitive electronics.

Simulation Profile Creation

The second element introduces simulation profiles with a demonstration profile based on Falcon 9 launch data. This design incorporates time-dependent acceleration, vibration, and shock

profiles that can reflect real world scenarios such as the Falcon 9 launch. By simulating dynamic mechanical loads, the tool can better evaluate how whiskers move and collect under varying stress, leading to improved bridging probability estimates. The input into the profiles will be the Monte Carlo distribution settings, acceleration, vibration, shock, and PCBs. With a standard profile, the output of different PCBs can be compared for more optimal design analysis. A custom script applies acceleration, vibration, and shock over time as well as acceleration direction. The demonstration profile can be used, or custom profiles can be created for standardized use on uploaded PCBs. The acceleration magnitude over time will be input as a function, and acceleration direction will be a function of board rotation in degrees as it is already implemented in the simulation. Vibration over time will also be input as a function of amplitude and frequency, with the possibility of multiple vibration implementations that can be analyzed using a Fourier Analysis. Shock values will be input as functions of half cycle periods and amplitude. The resulting vibration and shock will then be applied laterally as an applied acceleration vector to vary the motion of the whiskers.

Another input option is an interactive graph that lets users view and modify parameters like acceleration magnitude, direction and vibration frequency where they can toggle this module to compare static versus dynamic scenarios. The output of the simulation profiles will be the statistical data of whiskers and bridged connectors as already implemented with heatmaps of bridged conductors, whisker collections, and even tracking of whiskers over time. This module, entirely written in C# and built on Unity's physics system, is easily enabled or disabled to maintain compatibility with simpler testing scenarios, while greatly enhancing mechanical realism for aerospace or high-vibration applications.

Falcon 9 Launch Demonstration Profile

The demonstration profile is of the launch sequence of the SpaceX Falcon 9 rocket. The profile uses the acceleration, sinusoidal vibration, and shock data from the rocket lift off to the payload deployment. The acceleration magnitude of the rocket is a two-stage rocket ignition leading to payload deployment. The acceleration direction is trajectory of the rocket with the effects of acceleration and gravity acting upon the payload. The vibration data used is random vibrations above the natural frequency of the spacecraft. There are four shock events during the launch sequence, release of launch vehicle, stage separation, fairing deployment, and payload separation [8]. MATLAB was used to create representations of acceleration, vibration and shock values. The figure below shows the acceleration profile of Falcon 9 launch.

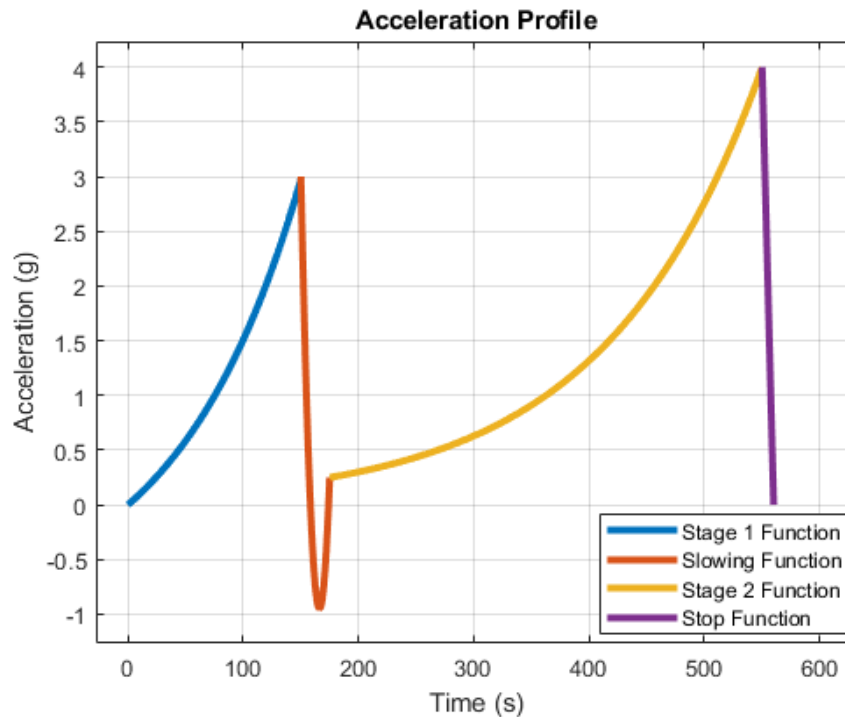


Figure 10: Falcon 9 Acceleration Profile

The vibration data consists of three randomly chosen vibration values that are within Falcon 9 user guide parameters and combined.

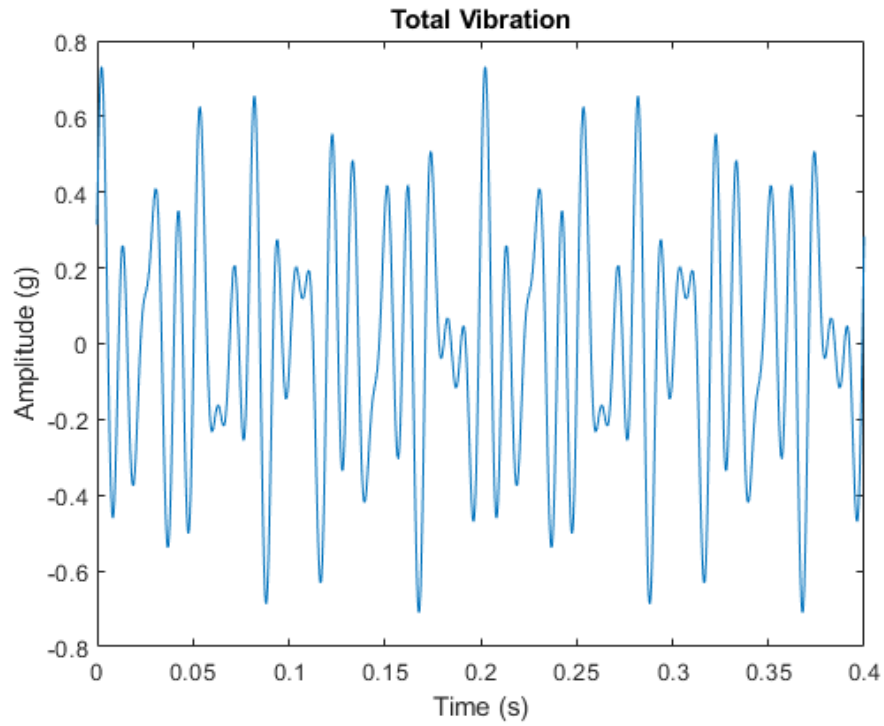


Figure 11: Sinusoidal Vibration of Falcon 9 Launch

The shock events are shown in the figure below.

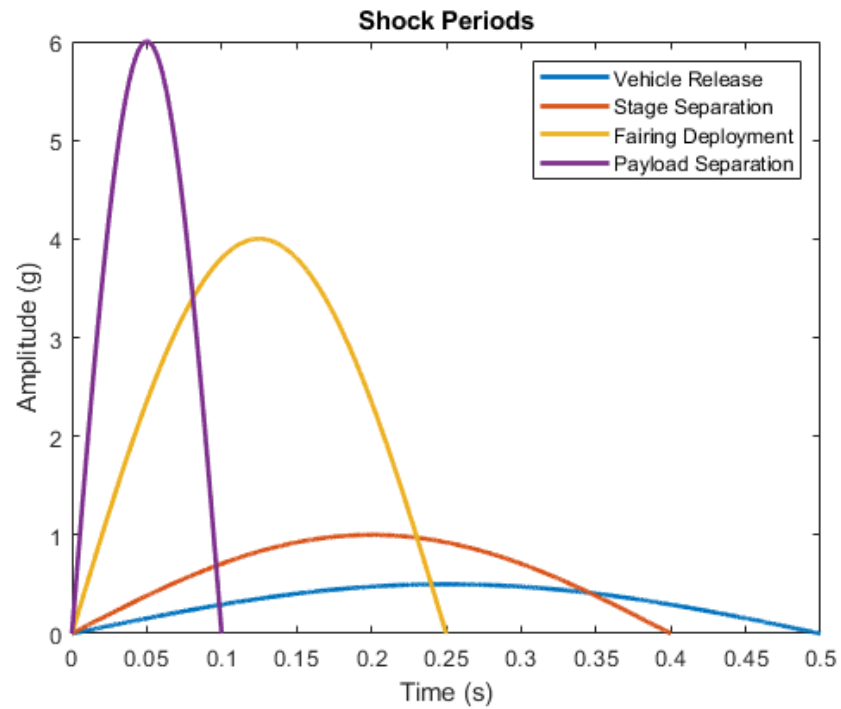


Figure 12: Shock Events of Falcon 9 Launch

PCB Model Importing

In the context of simulation analysis, importing different 3D circuit board models into Unity is an important task for engineers, developers, and designers to test and visualize interactive physical simulation, such as metal whiskers falling onto printed circuit boards (PCBs). This functionality is particularly valuable in assessing design robustness and anticipating failure modes without physical prototyping. In this case study, students explored various tools and methods to successfully import and integrate circuit board models into Unity using a combination of KiCAD, Fusion 360, and SolidWorks.

The process began with sourcing the necessary CAD models from online repositories. Although Altium Designer is typically regarded as the industry-standard tool for PCB modeling due to its advanced design features and compatibility, due to the access limitations, it led to the adoption of KiCAD as an alternative. KiCAD proved to be a functional and accessible open-source solution for this application, as illustrated in Figure 13.

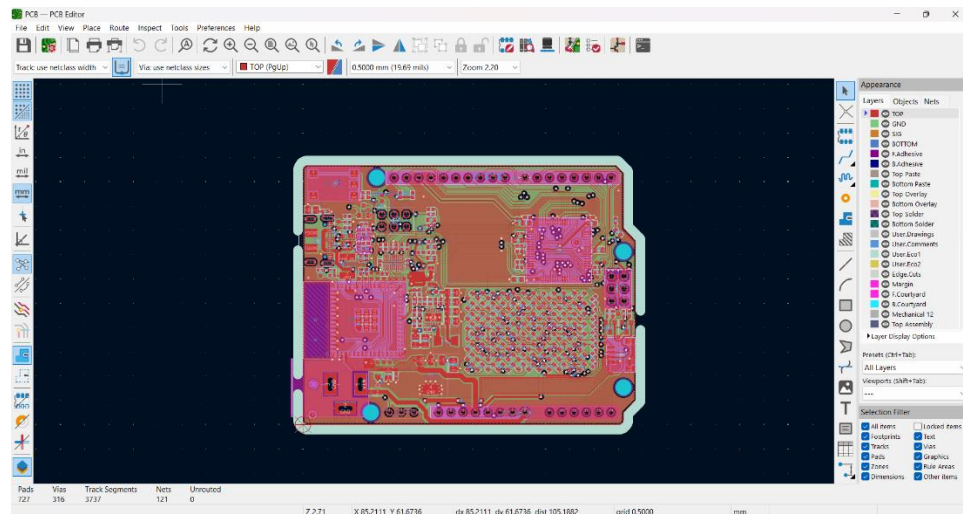


Figure 13: PB CAD File in KiCAD

Once the CAD file was opened in KiCAD, the model was exported in the .STEP file format. This format was chosen for its compatibility with Fusion 360, a platform capable of

generating accurate and realistic 3D models. During the export process, it was critical to avoid enabling the “Fuse Shapes” option, which it will significantly increases computational load and may result in system instability or crashes. Figure 14 and Figure 15 respectively depict the export interface and the disabled Fuse Shapes configuration.

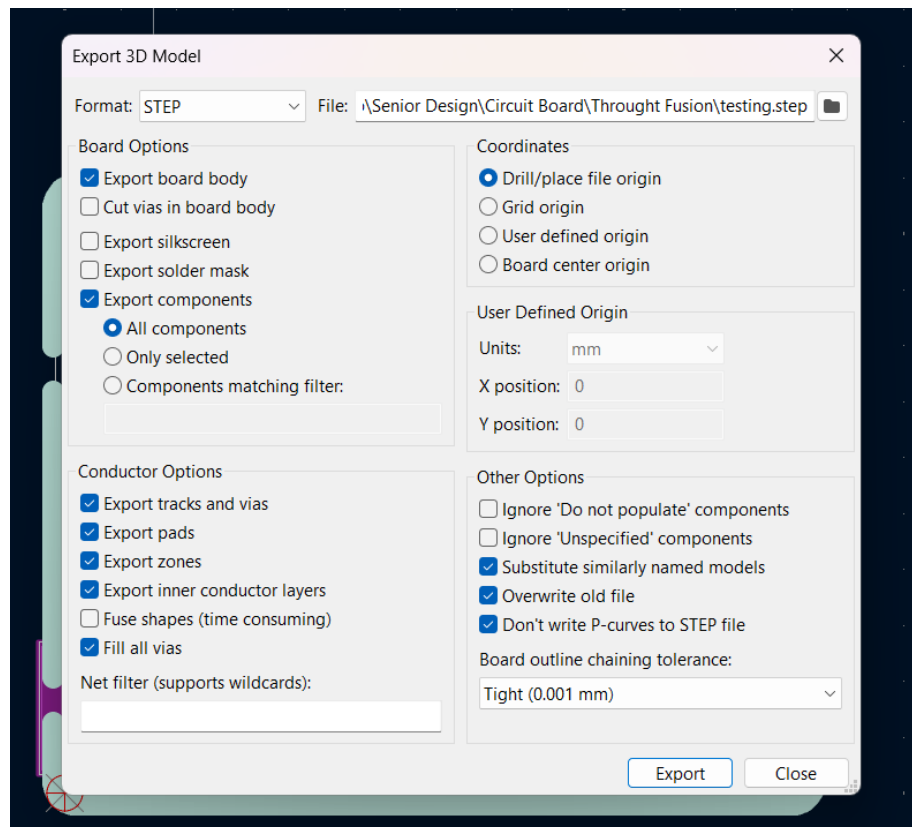


Figure 14: PB Export Interface

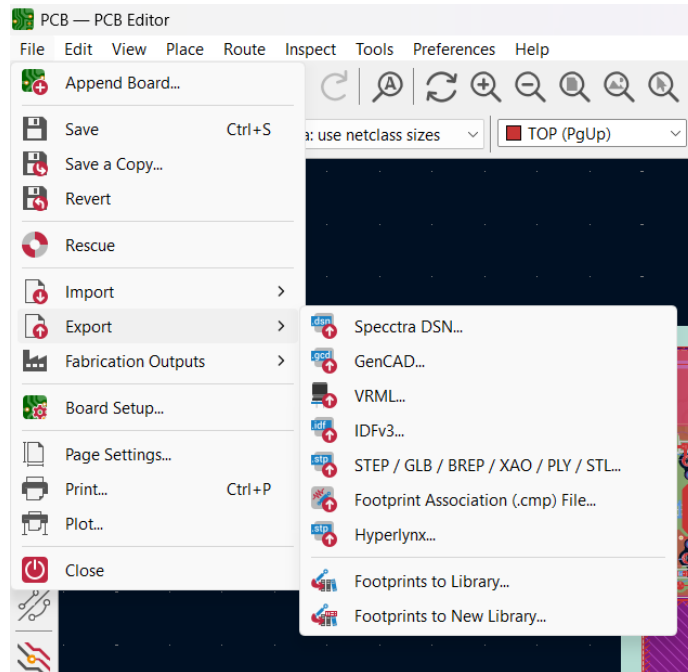


Figure 15: Disabled Fuse Shapes Configuration

The exported .STEP file was then imported into Fusion 360. Upon loading, Fusion 360 constructed a detailed 3D structure of the PCB model, as shown in Figure 16.

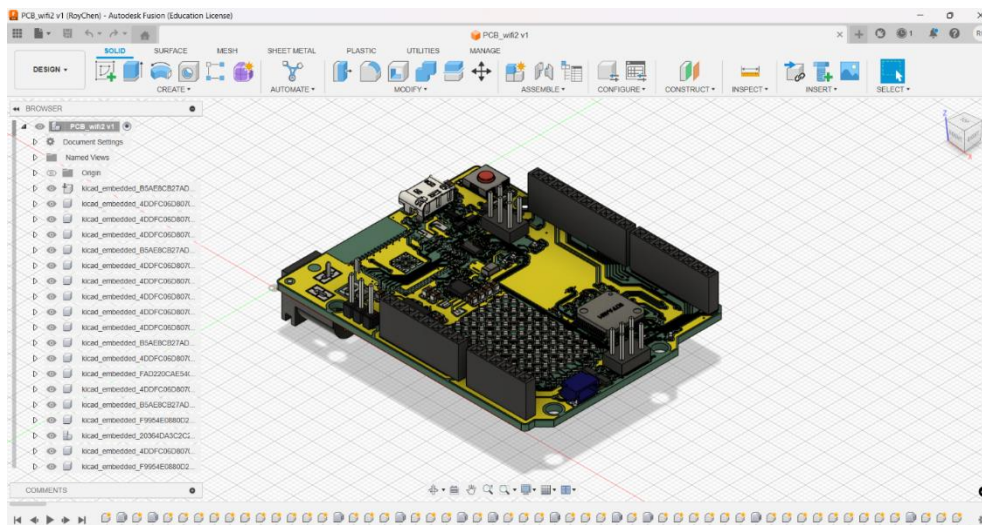


Figure 16: Fusion360 PCB Model

This model was then exported as an .OBJ file, a format recognized by Unity game engine. Exporting in this format ensured that both geometry and material data were preserved, as demonstrated in Figure 17.

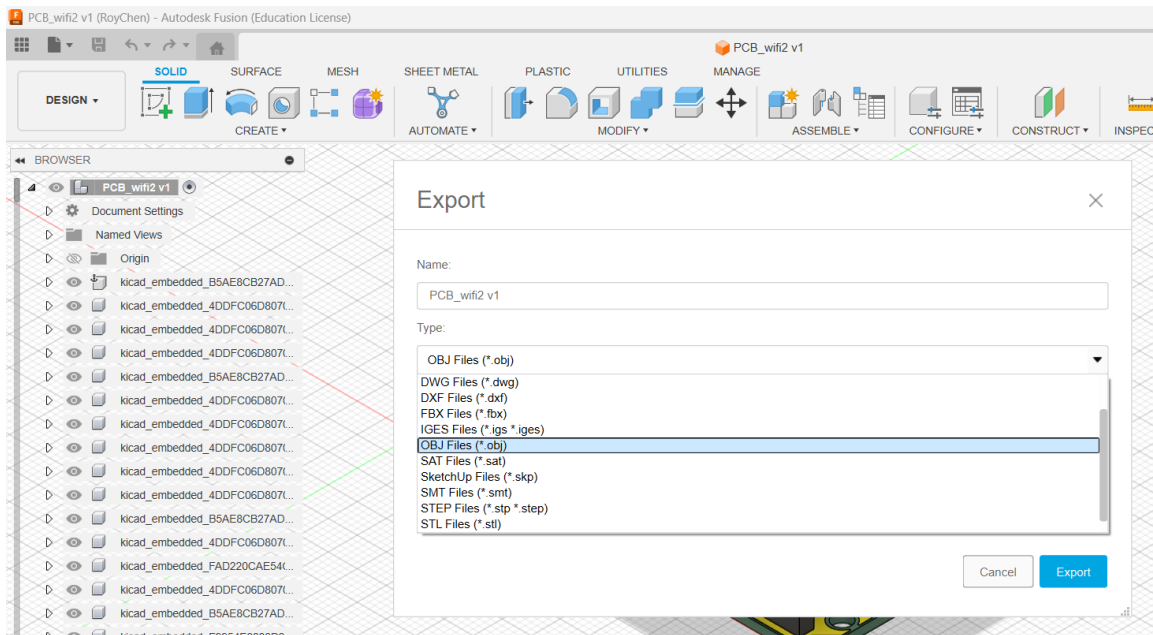


Figure 17: Exporting as .obj File

Following this, the Unity environment was prepared for model integration. The .OBJ file was imported into the project's asset and Boards, then drag the model within the simulation scene, as shown in Figure 18.

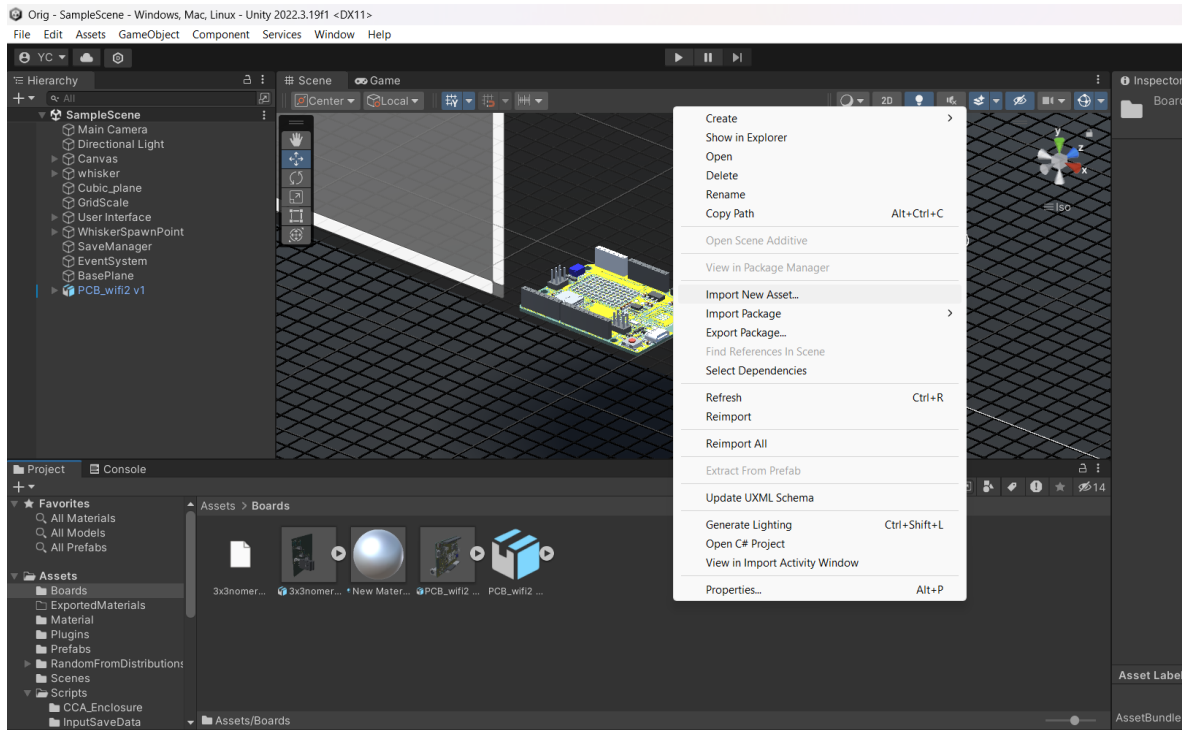


Figure 18: PCB .obj File in Unity

The imported object was then configured through the Hierarchy and Inspector panels. The model was assigned the tag “CircuitBoard” to facilitate interaction tracking during simulation. Adjust parameters such as position, rotation, and scale were adjusted to align the model within the scene environment (Figure 19). To recognize circuit board during the simulation, the Read/Write option in the asset import settings was enabled (Figure 20).

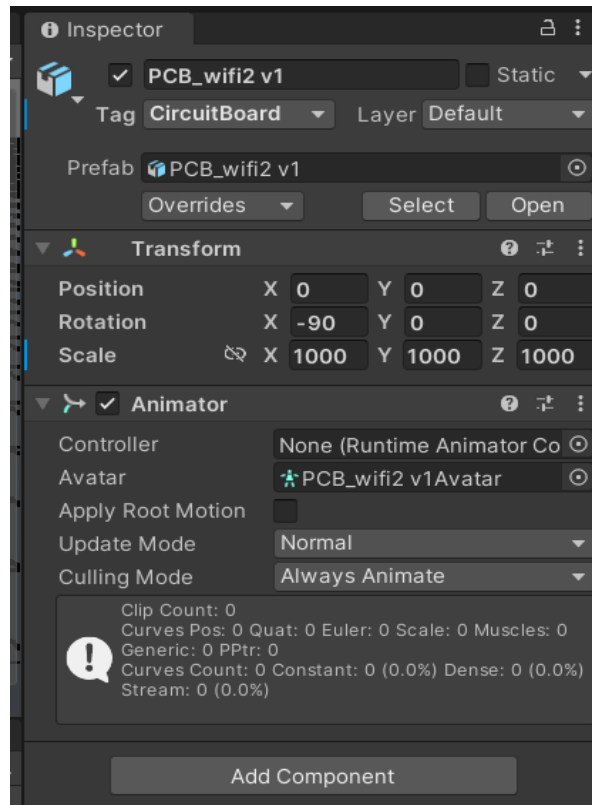


Figure 19: Unity Inspector Tab

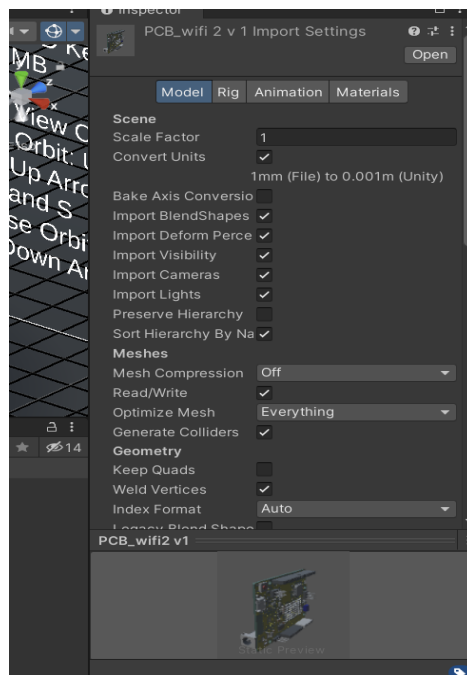


Figure 20: Unity Model Panel

Once the model was correctly placed and configured, the simulation process can be started. In Game Mode, make sure play size is Maximized, the Whisker Property checkbox was activated, allowing whiskers to interact with the PCB model surface. Special attention was given to aligning the whisker spawn area to the model's physical geometry to ensure visual and functional accuracy. Figure 21 illustrates the final alignment setup.

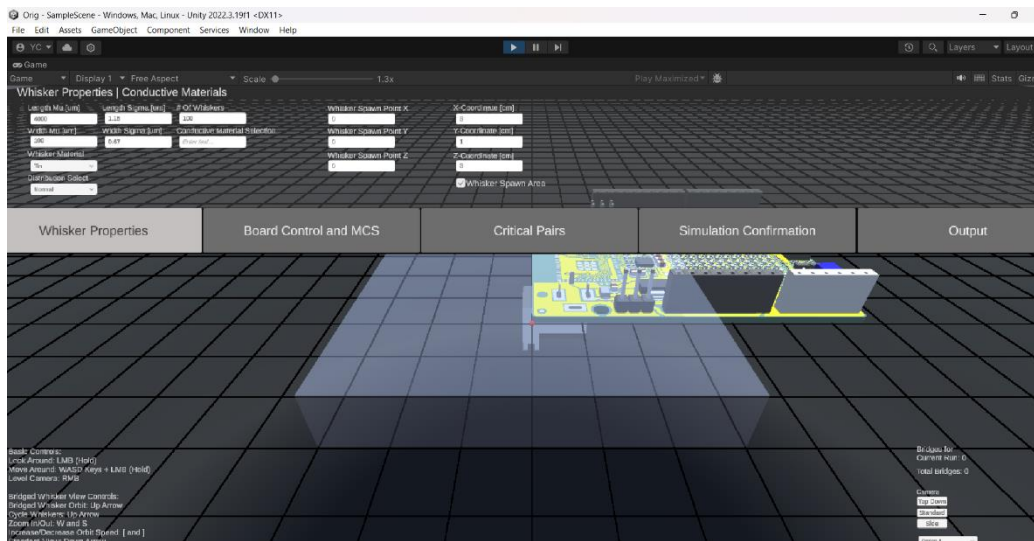


Figure 21: Unity In-Simulation Model

The completed simulation setup successfully rendered the circuit board as an interactive GameObject within Unity. The model maintained its geometric integrity and responded correctly to simulation triggers, such as whisker collisions and spawn behavior. This outcome is presented in Figure 22.

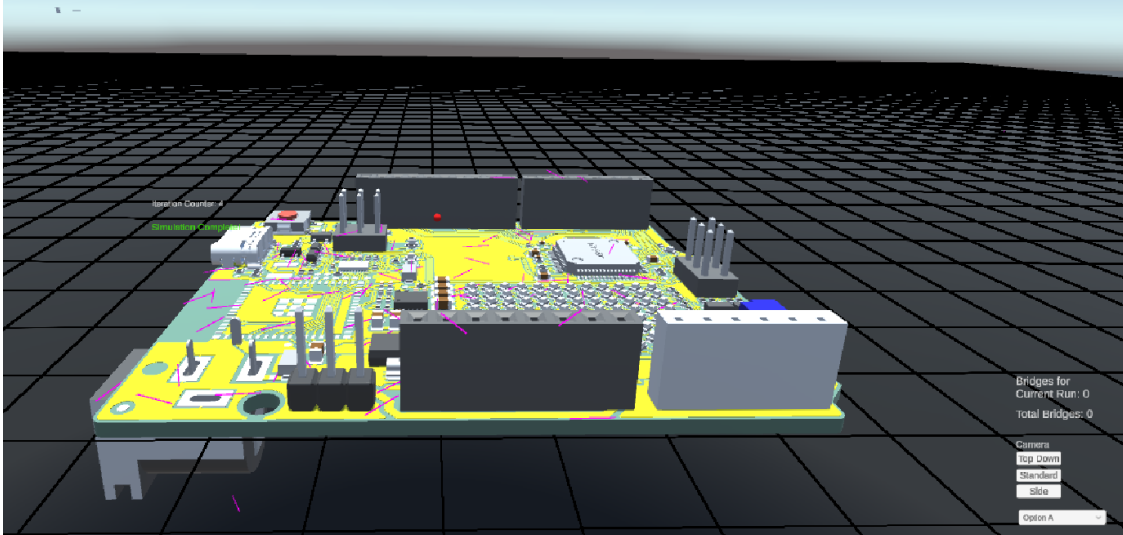


Figure 22: Whiskers on Imported PCB

Each design element was evaluated using a decision matrix that considered feasibility, alignment with our problem statement, and the time required to implement the solutions. The results indicated that the electrostatic attraction module, while moderately complex, is highly relevant for predicting bridging events; the Falcon 9 acceleration profiles, although requiring more development time, offer substantial improvements in modeling aerospace conditions; and the seamless PCB import module is rated highly for usability, ensuring that real PCB designs can be tested with minimal manual intervention. Based on these evaluations, our final approach integrates the three elements in phases: first implementing seamless PCB import to ensure real geometry is preserved, then incorporating electrostatic attraction to capture short-circuit risks from charge buildup and finally integrating acceleration profiles to model demanding mechanical environments.

Subfunction Decision Making

This multi-faceted approach not only meets our design requirements for user-friendliness and simulation accuracy but also creates a robust and practical tool for engineers to evaluate tin whisker bridging risks under both standard and extreme conditions.

The table below outlines the types of simulation inputs used for different physical subfunctions affecting a printed circuit board (PCB):

Table 4: Subfunction Decision Matrix

Subfunction		
External Force	Electrostatic Force	Temperature Fluctuation
Simulation Profile	Time-Function Input	Graph Input
PCB Input	Altium	KiCad

- **Subfunction** lists the external influences simulated: external mechanical forces, electrostatic forces, and temperature fluctuations.
- **Simulation Profile** indicates how each subfunction is modeled:
 - *Time-Function Input* means the force varies over time (like a dynamic load).
 - *Graph Input* means the electrostatic force is input based on a predefined graph or dataset.
- **PCB Input** specifies the software used to import or design the PCB layout for each case:
 - *Altium* is used for external force simulation.
 - *KiCad* is used for electrostatic force simulation.

In summary, these three design elements- Electrostatic Attraction, Simulation Profiles, and Seamless PCB Import- collectively meet the design requirements for user-friendliness, simulation accuracy, and real-world application. By combining them within Unity's C# environment, the final simulation tool can capture both internal whisker dynamics (electrostatic

forces, bridging events) and external environmental stresses (launch acceleration, shock, or handling). This synergy ensures an advanced, high-fidelity simulation platform suitable for a wide range of PCB reliability assessments.

CHAPTER 5

DEVELOPMENT, TESTING, AND RESULTS

Simulation Profiles Addition

The first step in the simulation profiles was to add an input for the time length of each simulation iteration. The previous team had established a framework for time length adjustment and only required creating an input field to apply the user input time length. This time length would affect each operation of the simulation.

Vibration and Shock Time Activation

The next step was implementing the vibration and shock forces at a specified time during the simulation. Both vibration and shock forces were activated by a button when the user would click to activate the function. To activate over the timeframe of the simulation, a start time input was created for each function. The functions were then set to be called from the simulation time loop within the C# Script 'UI Script' when simulation time reaches their respective start times. Vibration needed no other changes as users can input Amplitude, Frequency, and Duration. Shock was given an input field for the $\frac{1}{2}$ sinusoidal period representing one shockwave over time where the frequency can then be calculated instead of being hard coded as the previous team had done.

An initial test after the function changes required a variable for each function to only activate once per each simulation iteration. This makes each function only operate once but can be configured for more repetitions in future development. Currently, the Confirmation button for each function is still operational for checking inputs, but the start time inputs should be used for simulation operation. The figure below shows the input fields for the vibration and shock functions.

Vibration	Enter text...	Enter text...	Enter text...	Enter text...
	Amplitude [N]	Frequency [Hz]	Duration [s]	Vibration Start [s]
Shock	Enter text...	Enter text...	Enter text...	
	Amplitude [N]	Shock Start [s]	1/2 Period of Sine[s]	

Figure 23: Vibration and Shock Time based Inputs

Time-Dependent Acceleration

To develop more real-world possibilities, the next step was to create an input for time-dependent acceleration on the whiskers. The simulation prior to changes had input for a constant force to act as gravity. After some initial testing of its capabilities, it was decided to replace the gravity system with the new acceleration input system as it could not support function input, updating of force vectors, and was buried within other code. The acceleration input system began with a model project for simpler implementation.

Acceleration Model Development

The model was created to follow the same process as the tin whisker simulation. The initial model was to test the feasibility and method of time-based acceleration. It consisted of a ball “GameObject” in the middle of the chamber that has an input for the acceleration of the vertical direction of the ball. This input required a specialized code to parse the string input into math functions and output as C# and Unity could not do this natively. The chosen math parser used is the mXparser downloaded from GitHub as it is a robust option for various math functions. A new script called “FunctionInputHandler” was created to take a function input and time input to output an acceleration value. This acceleration value would then be applied to the “Rigidbody” or physics system of Unity acting on ball in the form of acceleration. This method works for changing the acceleration, but there is some error likely in the resolution of acceleration updating.

The problem that arises is that a higher rate of acceleration updating can be computationally expensive on the computer. This method was the simplest to implement and is capable of change in future development. The tin whisker simulation uses a button to spawn the whiskers with no force affecting the spawned whiskers, then applying the force when the simulation is activated by another button. The model was then developed to further match this process.

The final model version included a button to spawn a ball “GameObject” and a separate simulation start button. There were also inputs for function input and time length input. In the previous model, a ball was already present in the simulation and could simply attach the acceleration function to the ball game object and activated on program start. The final model required instances of the ball GameObject to be spawned, simulating whisker spawns. An acceleration script was used to call the function handling script at each time to update the Rigidbody physics of the ball. The acceleration script needed to be applied to each ball instance as it spawned. The function was then deactivated until the simulation was started through a variable. This process developed on the model was successful in applying time-dependent acceleration on spawned ball game objects during the simulation time. The below figure shows the final model for testing the time-dependent acceleration system before applying it to the simulation.

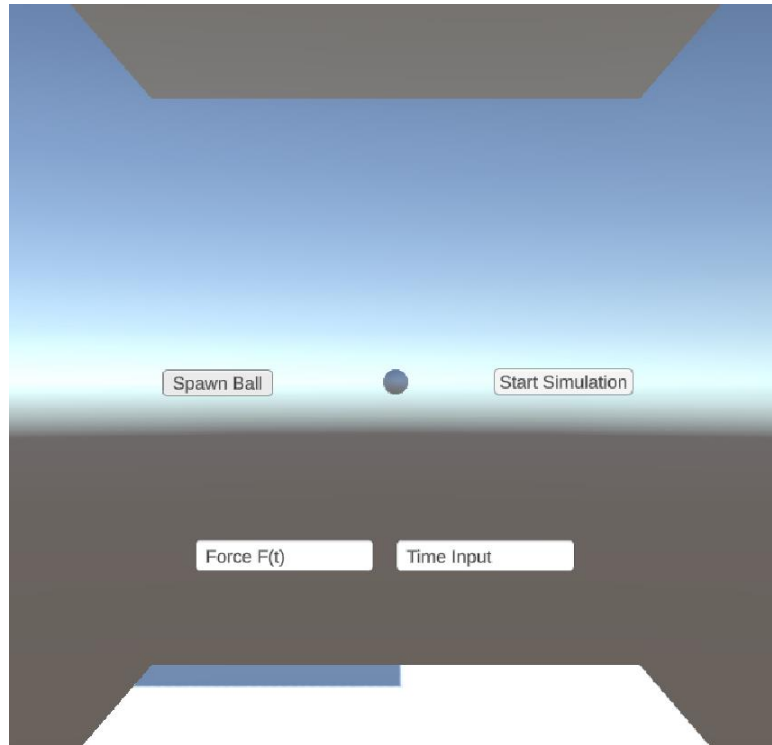


Figure 24: Time-dependent Acceleration Model

Simulation Acceleration Implementation

The process was applied by attaching the whisker acceleration script to the whisker game object each whisker clone is copied from. This is the function that updates the acceleration force and applies it to the Rigidbody. The “UI Script” contains the spawning of the whisker clone. The whisker acceleration script is attached and disabled on the spawn. As noted before, the UI Script also contains the simulation timeline and reset functionality. The acceleration function is enabled during the “ReloadWhiskerButton” function for resetting acceleration for each iteration. The “WhiskerControl” script contains the function for the simulation start. The acceleration force is applied, and the start time is set for each whisker clone by this function.

The acceleration functionality of the simulation is capable of time-dependent acceleration using various math functions. Acceleration direction can be altered using the input to rotate the

board to a static rotation or constant angular velocity. The simulation is limited on number of whiskers due to the numerous calculations done, causing lag based on the user's computer. There are possible less complex processes such as creating an array of accelerations over the time.

Profile Saving

Input save functionality was already available from the previous team's development. The inputs added are simply added to the save function. Basic Falcon 9 Launch profile is set to be the default values when the simulation is open. User created inputs can be saved and loaded using this functionality.

Adding Electrostatic Attraction

Equation/Setup

The process of adding electrostatic attraction to the Unity Simulator followed a certain order to ensure the smooth integration of a new function to the previous teams' work. The first function that was added was inputting a force that would attract the whiskers to a fixed position. The force will be modeled using the electrostatic force equation, as shown below:

$$F = k_e \frac{q_1 q_2}{d^2}$$

Where F is the force, k_e is Coulomb's Constant, q_1 and q_2 are the signed magnitudes of the charges, and d is the distance between the charges. With the force's magnitude being inversely proportional to the distance, the charges would experience more force going towards each other the closer they are.

Constant Position/Magnitude

Before adding in any electrostatic code in C#, a Boolean function, which allows the user to select if something was on ("true") or off ("false"), was added as "applyElectrostaticForce" in the WhiskerControl script of the Unity simulator. The equation was added in the Unity code as

having the product of k_e , q_1 , and q_2 be modeled as one value known as “electrostaticConstant”, which allowed for ease of manipulation to see the results of the code. From here, a value “electrostaticTarget” would be used as the point in the simulation where the whiskers will be attracted to, with the fixed point being at (0,0,0) at first for testing.

In the simulation, the whiskers will continue to approach the source of the attraction, eventually touching it if it is possible. However, with the code, if a whisker were to “touch” the point of attraction, the simulator would crash, the distance between the whisker and the point would be 0, which would involve dividing by 0 (as per the electrostatic force equation). To solve this, a new value was added known as “minDistance”, which would make it so the d value could not be less than said value, which meant that it could not be 0.

Changing Position/Magnitude

From there, the next step from having one constant magnitude as “electrostaticConstant” pointing towards one constant point “electrostaticTarget” was to make it so the user could change those values without having to go into the code. This was done by the function “TMP_InputField”, which is a C# function in Unity that allows a user to input something as the simulation is running.

In this case, the input fields added was “electrostaticConstantInput” to change the electrostatic constant, as well as “electroTargetX”, “electroTargetY”, and “electroTargetZ”, which would allow the user to change the target of where the whiskers would experience a force to. After adding that to the code, the Unity simulator would be updated so that these values could be changed while the simulation is running as part of the UI.

After that, the goal was to be able to make it so that the user could change the number of how many electrostatic forces were active at once, with the user also being able to change the

constants and targets of each. This involved the “TMP_InputField” function again, this time alongside a new value, “electroCountInputField”. What this value would do is check a user input number for how many electrostatic forces should be active at once.

The UI for electrostatic forces, which has up to five different fields for these forces and their attraction points, would allow the user to fill out as many of the five fields they would need, with the code only reading an amount equal to the value input for “electroCountInputField” (up to a maximum of five). Five were chosen so that the UI would not become cramped, as all these fields were added to the Whisker Control UI tab. This is shown below in Figure 25 as part of the Whisker Properties tab.

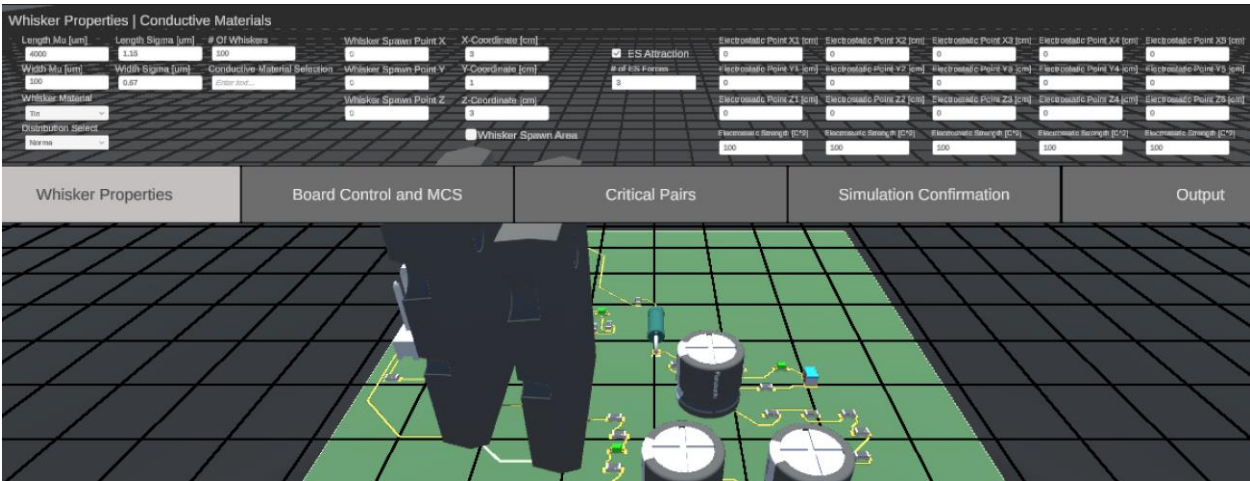


Figure 25: Whisker Properties Tab with Electrostatic Attraction

Figure 26 below offers a better look at what has been added to the Whisker Properties tab.

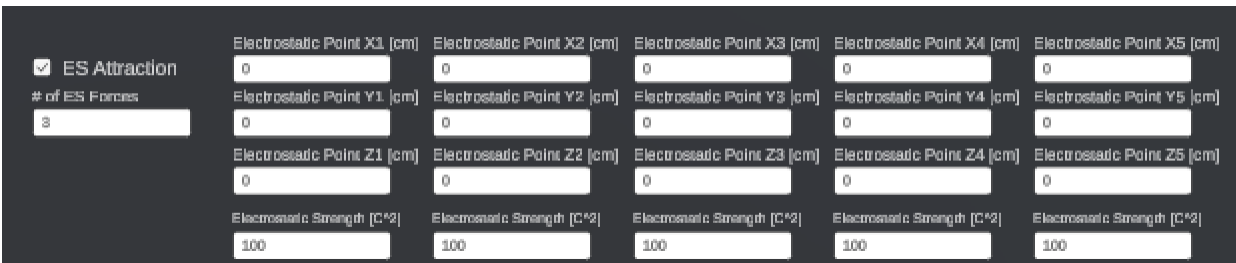


Figure 26: Electrostatic Attraction UI

Heatmap Improvement

Testing

We spent the majority of our development time attempting to implement a functional heatmap system in Unity that could visualize bridged whisker pairs across the surface of a PCB. While we did not achieve a complete or functional final product, we tested multiple implementation strategies and made significant progress in refactoring and debugging.

Our initial approach colored individual conductor GameObjects when a whisker bridge occurred. However, this was ineffective for showing overall bridge activity because only specific objects changed color, and visual clarity was limited. To improve this, we tested a new approach: dynamically generating a Texture2D and mapping it onto a single flat plane (the PCB) to give a continuous red-green gradient view.

Original Code:

```
using System.Drawing;
using System.Numerics;

public void GenerateHeatmap(List<Vector3> bridgePositions)
{
    foreach (var pos in bridgePositions)
    {
        GameObject hitObj = GetObjectAtPosition(pos);
        if (hitObj != null)
        {
            Renderer rend = hitObj.GetComponent<Renderer>();
            if (rend != null)
                rend.material.color = Color.red;
        }
    }
}
```

This method relied on detecting the exact conductor at a position and changing its material color, which was visually fragmented and failed if the conductor wasn't directly hit or found.

New Code:

```
using System.Drawing;
using System.Numerics;
using System;

public void GenerateHeatmap(List<Vector3> bridgePositions)
{
    heatmapTexture = new Texture2D(textureResolution, textureResolution,
TextureFormat.RGBA32, false);
    colors = new Color32[textureResolution * textureResolution];

    for (int i = 0; i < colors.Length; i++)
        colors[i] = Color.green;

    foreach (var pos in bridgePositions)
    {
        int x = Mathf.FloorToInt((pos.x + 0.5f) * textureResolution);
        int y = Mathf.FloorToInt((pos.z + 0.5f) * textureResolution);
        int index = y * textureResolution + x;

        if (index >= 0 && index < colors.Length)
            colors[index] = Color.red;
    }

    heatmapTexture.SetPixels32(colors);
    heatmapTexture.Apply();
    pcbMaterial.mainTexture = heatmapTexture;
}
```

This version created a unified heatmap by translating position data into pixel space. Although the logic made sense conceptually, we never fully got it working — mostly due to misalignment between world space and texture coordinates.

We also heavily tested our UI logic for toggling the heatmap on and off. Here's the toggle function we tested repeatedly:

```
public void heatMapButton()
{
    if (toggleColorOn)
    {
        heatMapToggleOFF();
        HeatUI.SetActive(false);
    }
    else
    {
        GenerateHeatmap(whiskerBridgePositions);
        HeatUI.SetActive(true);
        toggleColorOn = true;
    }
}
```

This logic handled activating the UI and generating the heatmap, but issues arose from inconsistent state handling, missing references, and unclear GameObject visibility control.

Discussion

Despite our efforts, the heatmap feature remained incomplete by the end of the project. We believe there are several reasons for this:

- **Coordinate space mismatch:** Mapping real-world object positions to a 2D texture space was more complex than expected, and we lacked a solid reference system for normalizing and scaling coordinates.
- **Broken Unity links:** Renderer and material references often got disconnected in the Unity editor, preventing textures from showing at all.
- **Scene interference:** Other GameObjects and leftover logic from earlier prototypes interfered with the heatmap script. We spent time stripping the scene down to isolate the issue, but by then it was late in development.
- **Limited time for debugging:** We devoted most of our time to testing and rewriting logic, leaving little room for polishing or validating the final output.

Conclusion

Even though the final product wasn't functional, we gained valuable experience debugging real-time rendering systems in Unity, writing texture-based visualizations, and organizing UI-state-driven logic. We learned how fragile GameObject references can be, and how important it is to plan coordinate systems early when building custom visualization features. With more time and a simplified testing environment from the beginning, we believe the heatmap system could have been completed.

PCB Import

Testing Phase for Importing PCB

The initial testing phase involved evaluating several software tools for their ability to generate and export 3D PCB models suitable for Unity. The software considered included:

- **Altium Designer:** Recognized for its robust PCB modeling capabilities. However, access was restricted due to licensing constraints, preventing further testing.
- **KiCAD:** Chosen as a viable alternative due to its open-source nature and sufficient export functionality.
- **Fusion 360:** Used for refining and converting 3D models into Unity-compatible formats.
- **Blender:** Attempted to assign material properties for Unity integration. However, Unity failed to recognize Blender's material assignments, resulting in grey, texture less models.
- **FreeCAD:** Tested but found to be less streamlined in workflow compared to Fusion 360.
- The key limitations that were found during testing mainly included Blender's incompatibility with Unity's material recognition

While the visual aspect of the model was preserved, the simulated process failed to detect short circuits resulting from whiskers falling onto the board. One of the hypotheses is that the complexity of the real-life circuit board is causing confusion to the code's detection section within Unity. This issue suggests a potential need to simplify the mesh or optimize the object's physical properties in Unity.

Another issue encountered was the orientation of the circuit board within Unity. The previous team created the circuit board model within Unity, placing the origin point at the center of the board. During the process of importing a realistic new board into Unity, the results in an origin point that varies depending on the original CAD drawing of the board. Accordingly, additional adjustments are required to properly orient the board, align the whisker spawn area, and scale the board to the correct size.

The final notable limit was the multi-step import procedure, which required the use of two or more separate engineering software. This not only increased the chance of model corruption or data loss during the transporting process but also made the workflow inefficient and less user friendly.

CHAPTER 6

CONCLUSION

The 3D Whisker Model Tool was already a capable tool of statistical analysis in predicting whisker conducting situations. Our goal was to develop the analysis capabilities for more real-world applications. The goal was to create a standardized simulation that can be applied to different PCBs, creating a reliable testing environment. The simulation was developed in three major areas, the PCB Importing process, the external forces development, and the creation of standardized testing profiles including Heatmap improvements.

While importing 3D PCB models into Unity was established, several areas require improvement, particularly in script integration and workflow efficiency. Future work may involve developing a direct KiCAD-to-Unity plugin or implementing automated mesh optimization to reduce processing overhead. Addressing these challenges is essential for achieving real-time interaction and testing reliability in simulation environments.

Electrostatic attraction forces were developed to further ground the simulation in real-world environments. Whiskers will be naturally attracted to things that are electrically charged if they are charged themselves, which happens due to its environment being a PCB. Not only that, adding attraction/repulsion forces could be used to design a “trap” for whiskers, since the whiskers will be congregating at a certain spot that the electrostatic attracting force is at.

Simulation profiles were the largest step towards standardized scenarios. It included the transition from the constant force gravity system to the function input time-dependent acceleration. The current vibration and shock forces are also developed to work on the simulation timeline. All current inputs into the simulation will be saved as a setting that can be

run again. A default input demonstrates a portion of the Falcon 9 launch that can be altered when the user defines.

Throughout the development process, we explored multiple approaches to modifying the existing heatmap script. Testing was conducted across various versions of the UI and shader materials, and several methods were implemented to try and capture whisker activity accurately. While we made progress in understanding Unity's C# scripting capabilities, we were ultimately unable to produce a fully functional or visual heatmap by the project's conclusion.

We encountered problems with the UI script frequently. The script responsible for toggling the heatmap on and off frequently failed to activate the visual elements properly. In many cases, the errors were thrown when attempting to update materials or textures, and material assignment bugs that caused the heatmap to remain inactive even when triggered through the interface. Debugging these errors consumed time as we attempted to track reference objects and synchronize script actions with UI interactions.

These technical difficulties reflect broader limitations when using Unity for engineering simulations. Unity provides a robust platform for 3D applications, but it is not created for physical simulation and data analysis. As an example, there is no graphing functionality native to the Unity engine so a math parsing tool and graphing script must be created to apply math function input capabilities. Even when the acceleration input functionality was added, the simulation had trouble with numerous whiskers likely due to the calculations required. These problems may be easier to avoid with more computer science background technical advising. Knowledge of the proper code formatting procedures would help with organization as the additional team efforts on code can make it difficult to understand the code layout and processes.

The 3rd Auburn Tin Whisker Model Team aimed to develop the already capable whisker analysis simulation into a more flexible real-world analysis tool. The challenge of importing PCBs has highlighted difficulties in the use of the 3D game engine, Unity. Electrostatic and Acceleration input additions have helped to develop a more robust whisker analysis tool. Future endeavors into tin whisker simulation should consider alternatives such as ANSYS or work in conjunction with technical support of computer science specialists.

APPENDIX A

PROGRAMMATIC QUESTIONS

Electronics are embedded in the majority control systems nowadays, so it is our professional responsibility as engineers to ensure safety to the best of our ability for the given application. Accordingly, identifying the failure modes that may occur as a designers and users with properly vetted and tested to assess to the probability of such failure modes is our responsibility for the team.

In terms of design constraints. There is no significant direct environmental impact, since the simulation is software based. From an economic perspective, our project was creating a simulation system to predict tin whiskers falling on the board to cost short circuits, therefore it reduces the risk of damage circuit board. While there is no direct impact on public health, safety or welfare, enhancing circuit reliability may benefit in safety-critical applications. At the end, no direct global, cultural or social impact was identified.

During this project, we gained hands-on experience in various engineering tools and concepts related to software and engineering development. We learned the fundamentals of C# programming and how to utilize the Unity game engine, including its setup and core properties adjustment. Also, we developed the ability to update interactive software environments and analyze how metal reacts to real-world situations such as acceleration, vibration, electrostatic forces, etc. In addition, we explored circuit design using KiCAD and 3D modeling by Fusion 360, both of which were critical in supporting the complete import process. This different experience enhanced our understanding and ability to combine theory and software together.

Since whiskers can grow under various environmental conditions, our project addresses a critical reliability concern in electronic systems. By simulating whisker growth and its

interaction with circuit boards, our project provides a valuable tool for predicting potential short circuits. This contributes to enhancing the safety, durability, and performance of electronic devices in real-world applications.

The project was entirely on our computer simulations, which meant that there were no economic, environmental, or ethical concerns, as everything that we did was technically theoretical, given that we were focused on simulating environments. There was nothing to physically build, since this was comprised entirely in C# coding and Unity.

APPENDIX B

Development Gantt Chart

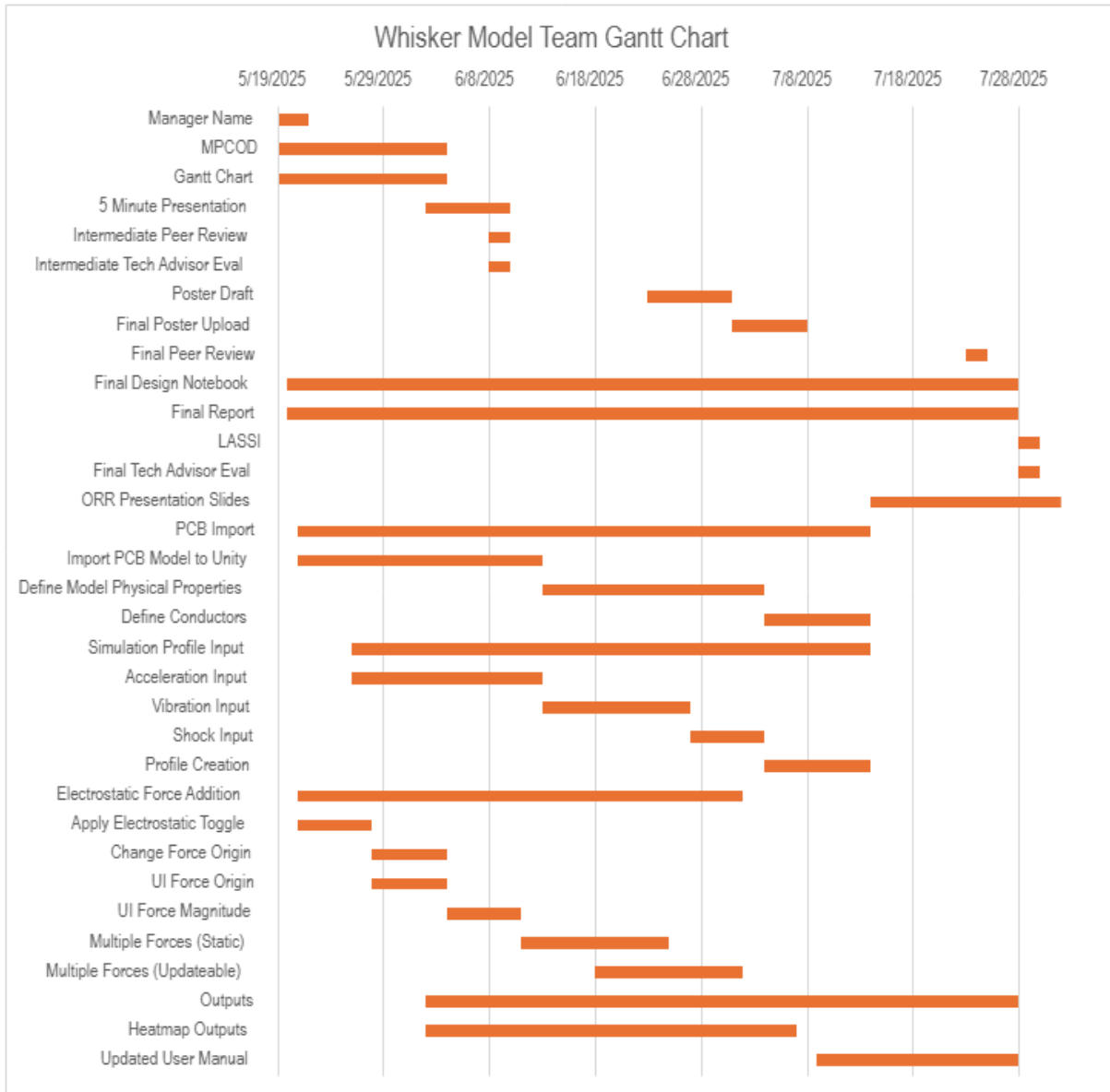


Figure 27: Whisker Model Team Gantt Chart

REFERENCES

- [1] Panashchenko, Lyudmyla. *The Art of Metal Whisker Appreciation: A Practical Guide for Electronics Professionals*. 2012.
- [2] “Basic Info on Tin Whiskers.” *Nepp.nasa.gov*, nepp.nasa.gov/whisker/background/index.htm.
- [3] Kenton, Will. “Monte Carlo Simulation.” *Investopedia*, 27 June 2024, www.investopedia.com/terms/m/montecarlosimulation.asp.
- [4] Leidecker, Henning, et al. *Tin Whiskers: A History of Documented Electrical System Failures a Briefing Prepared for the Space Shuttle Program Office*. 2006.
- [5] ipcb. “The Impact of Lead-Free Regulations on PCB Boards.” *Ipcb.com*, 2022, www.ipcb.com/pcb-blog/9653.html. Accessed 27 Apr. 2025.
- [6] “Tin Whiskers: Predicting the Risk | the Aerospace Corporation.” *Aerospace Corporation*, 24 June 2019, aerospace.org/story/tin-whiskers-predicting-risk. Accessed 27 Apr. 2025.
- [7] Knudson, K., et al. “Team 3: 3D Modeling of Detached Metal Whiskers, Senior Design Operation Readiness Report.” Auburn University, Dec. 2024.
- [8] SpaceX, “Falcon User’s Guide”, Sept. 2021.
- [9] *Arduino.cc*, 2024, docs.arduino.cc/hardware/portenta-h7/.